

Nama : Nindya Alif Romland

NIM : H1D024031

Shift KRS : A

Shift Baru : F

TUGAS JARINGAN SYARAF TIRUAN 2 PERTEMUAN 7

1. Import library

```
# Import library
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
```

- tensorflow digunakan sebagai framework utama untuk membangun dan melatih model jaringan saraf tiruan.
- Sequential digunakan untuk membuat model neural network secara berurutan layer demi layer.
- Dense digunakan untuk membuat fully connected layer, sedangkan Input digunakan untuk mendefinisikan bentuk input data.
- pandas digunakan untuk manipulasi data dan visualisasi history training dalam bentuk dataframe.
- numpy digunakan untuk operasi matematika dan manipulasi array.
- matplotlib.pyplot digunakan untuk membuat visualisasi grafik history training.
- seaborn digunakan untuk membuat visualisasi confusion matrix dalam bentuk heatmap.
- load_iris digunakan untuk memuat dataset Iris langsung dari sklearn tanpa memerlukan file eksternal.
- LabelEncoder digunakan untuk mengonversi label kategorikal menjadi numerik.
- train_test_split digunakan untuk membagi dataset menjadi data latih dan data uji.
- confusion_matrix digunakan untuk mengevaluasi hasil prediksi model terhadap label aslinya.

2. Load Dataset dan Preprocessing

```
# Muat dataset iris dari sklearn
iris = load_iris()
X = iris.data
y = iris.target

# Mengonversi label dari string menjadi numerik
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y) # Mengubah label jadi 0, 1, 2

# Memisahkan dataset menjadi data latih dan data validasi dengan rasio 80:20
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `load_iris()` digunakan untuk memuat dataset Iris yang sudah tersedia secara built-in di sklearn, terdiri dari 150 sampel dengan 4 fitur dan 3 kelas spesies bunga.
- `iris.data` digunakan untuk mengambil fitur berupa sepal length, sepal width, petal length, dan petal width sebagai input model.
- `iris.target` digunakan untuk mengambil label kelas yang sudah berbentuk numerik (0, 1, 2).
- `LabelEncoder().fit_transform(y)` digunakan untuk memastikan label dalam format numerik yang sesuai dan menyimpan mapping kelas agar dapat digunakan kembali saat prediksi data baru.
- `train_test_split()` digunakan untuk membagi dataset dengan rasio 80:20, di mana 80% digunakan untuk melatih model dan 20% untuk menguji performa model.

3. Pembuatan Model

```
model = Sequential([
    Input(shape=X_train.shape[1:]),
    Dense(1000, activation='relu'),
    Dense(500, activation='relu'),
    Dense(300, activation='relu'),
    Dense(3, activation='softmax')
])
```

Model dibuat menggunakan Sequential yang memungkinkan penyusunan layer secara berurutan. Arsitektur model terdiri dari:

- `Input(shape=X_train.shape[1:])` untuk mendefinisikan bentuk input sesuai jumlah fitur yaitu 4.
- `Dense(1000, activation='relu')`, `Dense(500, activation='relu')`, dan `Dense(300, activation='relu')` sebagai tiga hidden layer dengan fungsi aktivasi ReLU.
- `Dense(3, activation='softmax')` sebagai output layer dengan 3 neuron sesuai jumlah kelas dan fungsi aktivasi softmax untuk menghasilkan probabilitas per kelas.

Setelah model dibuat, `model.summary()` digunakan untuk menampilkan ringkasan arsitektur model.

4. Kompilasi Model

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Sebelum dilatih, model dikompilasi menggunakan fungsi `compile()` dengan tiga konfigurasi utama:

- `optimizer='adam'` sebagai metode optimasi adaptif yang menyesuaikan learning rate secara otomatis.
- `loss='sparse_categorical_crossentropy'` sebagai fungsi loss untuk klasifikasi multikelas dengan label integer.
- `metrics=['accuracy']` untuk memantau nilai akurasi selama training dan evaluasi.

5. Pelatihan dan Evaluasi Model

```
history = model.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=32,  
    validation_data=(X_test, y_test)  
)  
  
loss, accuracy = model.evaluate(X_test, y_test)  
print(f"Loss: {loss}, Accuracy: {accuracy}")  
  
pd.DataFrame(history.history).plot(figsize=(10,6))  
plt.show()
```

- `model.fit()` digunakan untuk melatih model menggunakan data latih selama 50 epoch dengan batch size 32, dan memvalidasi performa model menggunakan data uji di setiap epoch.
- `model.evaluate()` digunakan untuk menghitung nilai loss dan accuracy akhir model pada data uji setelah proses training selesai.
- `pd.DataFrame(history.history).plot()` digunakan untuk memvisualisasikan perubahan nilai loss dan accuracy selama proses training dari epoch ke epoch.

6. Prediksi dan Confusion Matrix

```
predictions = model.predict(X_test)  
  
# Mengambil indeks dari nilai probabilitas tertinggi untuk setiap prediksi  
predicted_classes = predictions.argmax(axis=1)  
  
print("Prediksi:", predicted_classes)  
print("Label Asli:", y_test)  
  
# Buat confusion matrix  
cm = confusion_matrix(y_test, predicted_classes)  
  
# Visualisasikan confusion matrix  
plt.figure(figsize=(8, 6))  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)  
plt.xlabel('Predicted')  
plt.ylabel('True')  
plt.title('Confusion Matrix')  
plt.show()
```

- `model.predict(X_test)` digunakan untuk menghasilkan probabilitas prediksi untuk setiap kelas pada data uji.
- `predictions.argmax(axis=1)` digunakan untuk mengambil indeks kelas dengan probabilitas tertinggi sebagai hasil prediksi akhir.
- `confusion_matrix()` digunakan untuk membandingkan hasil prediksi dengan label asli sehingga dapat diketahui jumlah prediksi benar dan salah pada setiap kelas.
- `sns.heatmap()` digunakan untuk memvisualisasikan confusion matrix dalam bentuk heatmap berwarna agar lebih mudah dibaca dan dianalisis.

7. Prediksi Data Baru

```
# Fungsi untuk memprediksi data input baru
def predict_new_data():
    sepal_length = float(input("Masukkan sepal length: "))
    sepal_width = float(input("Masukkan sepal width: "))
    petal_length = float(input("Masukkan petal length: "))
    petal_width = float(input("Masukkan petal width: "))

    # Membuat data array baru
    new_data = np.array([[sepal_length, sepal_width, petal_length, petal_width]])

    # Melakukan prediksi
    prediction = model.predict(new_data)
    predicted_class = prediction.argmax(axis=1)

    # Mengonversi hasil prediksi numerik menjadi label asli
    predicted_label = label_encoder.inverse_transform(predicted_class)
    print(f"Prediksi kelas: {predicted_label[0]}")

predict_new_data()
```

- Fungsi `predict_new_data()` dibuat untuk memungkinkan pengguna memasukkan nilai sepal length, sepal width, petal length, dan petal width secara manual melalui input.
- `np.array()` digunakan untuk mengubah data input pengguna menjadi format array yang dapat diproses oleh model.
- `model.predict()` digunakan untuk memprediksi spesies bunga berdasarkan data input yang dimasukkan pengguna.
- `label_encoder.inverse_transform()` digunakan untuk mengonversi hasil prediksi numerik kembali menjadi nama spesies aslinya.

8. Perubahan Data

Modul

```
# Muat dataset iris dari file CSV/data
dataset =
pd.read_csv('http://archive.ics.uci.edu/ml/machine-learning-database
s/iris/iris.data', header=None, sep=',')

# Menyusun data X (fitur) dan y (label)
X = dataset.iloc[:, :-1].values # 4 kolom pertama sebagai fitur
y = dataset.iloc[:, -1].values # Kolom terakhir sebagai label
```

Kode Google Collab

```
# Muat dataset iris dari sklearn
iris = load_iris()
X = iris.data
y = iris.target
```

Instruksi terbaru di Discord menyatakan bahwa untuk pertemuan 7 dataset tidak boleh menggunakan CSV, sehingga dataset Iris dimuat langsung dari library sklearn yang sudah built-in. Hasilnya tetap sama ada 150 sampel, 4 fitur, 3 kelas, hanya cara pengambilannya yang berbeda. Keuntungan pakai sklearn adalah tidak memerlukan koneksi internet dan tidak bergantung pada ketersediaan URL eksternal.

Selain itu ada tambahan import di kode kamu yang tidak ada di modul:

- `import matplotlib.pyplot as plt`
- `import seaborn as sns`
- `from sklearn.datasets import load_iris`
- `from sklearn.metrics import confusion_matrix`